MRTC 3D Campus Navigation

Shortest walking routes on a measured campus graph — Dijkstra's algorithm

Malla Reddy Vishwavidyapeeth – Technical Campus, Hyderabad

Team: _____ Hackathon: Algorithmic Problem Solving

Paradigm: Graph | Algorithm: Dijkstra's Shortest Path | Stack: React · TypeScript · Three.js

This PDF is slide content: each page = one slide (title, bullet points, visual idea, speaker notes).

The Problem

Finding your way on a large campus

- New students and visitors don't know which way is **shortest** between blocks, hostels, canteen and sports areas.
- Paths have **different lengths** — the route with fewer turns is not always the shortest.
- Wheelchair users need routes that **avoid stairs** — the shortest route may not be usable.
- Existing map apps don't know the **internal campus lanes** — Google suggests 550 m for a walk that is really 234 m.

SUGGESTED VISUAL

Photo 1 (blue A/B/C blocks) and photo 3 (football ground from the blocks).

Caption: "Where is the canteen from Block A — and what's the shortest way?"

SPEAKER NOTES

Open with a relatable moment: a first-year student at Block A who needs the canteen. Stress three things: distances differ, the shortest-looking path isn't always shortest, and accessibility matters. End with the Google 550 m vs real 234 m example — it shows why a campus-specific graph is needed.

Our Solution

A navigation system whose route comes from an algorithm — not a drawing



- **Data:** 18 places, 20 walkways — distances measured on Google Maps.
- **Brain:** Dijkstra finds the minimum-metres route (with accessibility rules).
- **Face:** Earth-to-campus 3D view, route animation, voice, live GPS / simulation.
- **Key principle:** the 3D scene only displays the algorithm's answer.

SPEAKER NOTES

This is the one-slide summary. Walk left to right through the pipeline. Emphasise that nothing about the route is hard-coded: change a distance in the data file and the route changes. The 3D view is the presentation layer; Dijkstra is the core.

Real Campus Data

Measured, labelled honestly

- Each walkway traced with Google Maps' **Measure distance** tool — along the path, not a straight line.
- Cross-checked with Google walking directions where available.
- Every value carries a status: **VERIFIED** / **ESTIMATED (map-measured)** / **UNKNOWN**.
- Missing distance → edge **disabled**, shown as "Measurement required". Never guessed.

Walkway	Distance
Block A – Block B	36.67 m
Block B – Block C	53.38 m
Block A – Block C (forecourt)	96.73 m
Block C – Canteen	84.78 m
Girls → Boys Hostel (via 4 segments)	234.23 m
Block C – Running Track – Football	52.88 + 11.33 m
Block A – Bike Parking	31.79 m
Block B – Car Parking	28.44 m

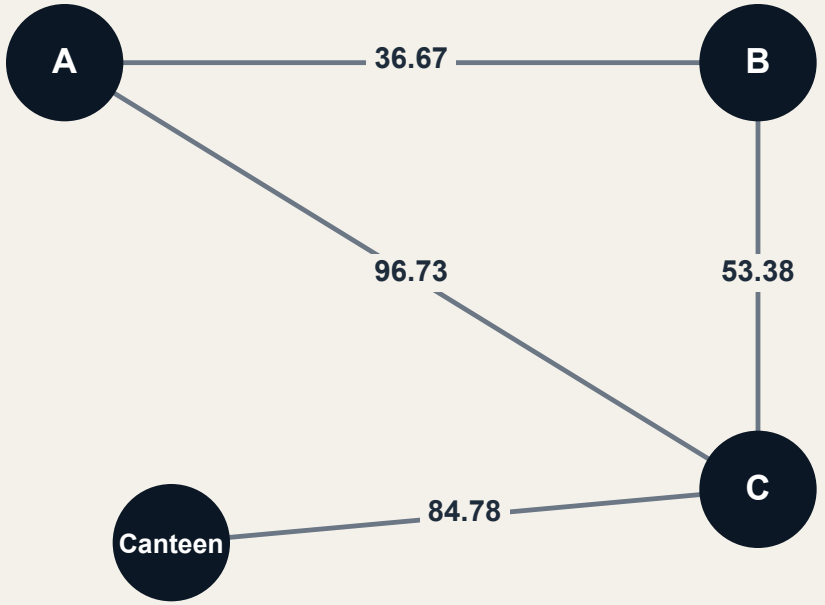
SPEAKER NOTES

Judges trust honest data. Explain how distances were measured and that all are currently ESTIMATED (map-measured, not walked). Mention straight-line vs path: Girls to Boys Hostel is 184 m straight but 234 m along real paths. Unknown values are switched off, not invented.

The Campus as a Graph

Places = nodes · walkways = edges · metres = weights

- **Node** (vertex): a place you can stand — entrance, court gate, road junction.
- **Edge**: a walkway that really exists. **Weight**: its length in metres.
- **Undirected** — paths work both ways. **Weighted** — lengths differ.
- **V = 18** nodes, **E = 20** edges. Stored as an **adjacency list**.
- No fake shortcuts: Girls ↔ Boys Hostel goes through 3 real junctions.



Subset of the real MRTC graph (metres).

SPEAKER NOTES

Define node, edge and weight using the diagram. Why adjacency list: 18 nodes and only 20 edges — a matrix would be 324 cells, ~88% empty. Dijkstra repeatedly asks 'who are my neighbours?', which a list answers directly.

Dijkstra's Algorithm — in Plain Words

"Friends on phones"

- Everyone holds a notepad: distance from the start. Start = 0, others = ∞.
- **Repeat:** pick the unfinished place with the **smallest** number — it is now **final**.
- It "phones" each neighbour: "my distance + path length — is that shorter than yours?"
- If yes, the neighbour updates its number and remembers **who called** (prev). This is **relaxation**.
- Stop when the destination is final. Follow **prev** backwards to get the route.

SPEAKER NOTES

Use the analogy before any code. Two words judges ask about: RELAXATION (a neighbour gets a shorter distance and updates) and FINAL (once the smallest unfinished number is picked, nothing can beat it because distances are never negative).

Worked Example: Block A → Canteen

Real MRTC distances, step by step

Round	Finalised	Updates (relaxation)	Notepads after
1	A (0)	B = 0 + 36.67; C = 0 + 96.73	B 36.67 · C 96.73
2	B (36.67)	C: 36.67 + 53.38 = 90.05 **< 96.73** → update	C **90.05** (via B)
3	C (90.05)	Canteen = 90.05 + 84.78	Canteen 174.83
4	Canteen	destination reached — stop	

Route (follow prev backwards): Canteen ← C ← B ← A, reversed: A → B → C → Canteen = 174.83 m

The direct A → C path (96.73 m) loses to A → B → C (90.05 m): more steps, fewer metres.

This is the project's milestone route from the specification.

SPEAKER NOTES

Walk the table row by row. Round 2 is the key moment: Block C improves from 96.73 to 90.05 — that is relaxation. Then show route reconstruction with prev pointers. Live demo: Algorithm panel → Play, then open the Debug log.

Data Structures & Complexity

Exactly what our implementation uses

Structure	Used for
Adjacency list (Map → edge list)	each place's neighbours
Binary min-heap (priority queue)	"which unfinished place is closest?"
Distance map	best known distance per place
Previous-node map	rebuild the route
Visited set	places already final

- Heap push/pop = **O(log V)** (tree height).
- ≤ V pops and ≤ 2E pushes →

Time O((V + E) log V)

Space O(V + E)

- **Lazy deletion:** outdated heap entries are skipped when popped.

SPEAKER NOTES

Where does log V come from? Only from the heap: it is a balanced tree about log2(n) levels tall, and push/pop walk one path. Honest footnote: the step-recording trace used for the visual panel adds O(V) per step; the core algorithm is O((V+E) log V).

Why Dijkstra? Compared on Our Real Campus

All 210 ordered pairs of places tested

Method	Optimises	Result on MRTC
Dijkstra	total metres	Always shortest (verified vs Floyd–Warshall)
BFS	number of edges	Longer in 70 / 210 pairs — A→C: 96.73 m instead of 90.05 m
Greedy (shortest next edge)	next step only	Wrong or stuck in 173 / 210 — A→Canteen dead-ends at the Football Court
DFS	nothing about length	finds a path, not the shortest
A*	metres + estimate	also optimal with a good heuristic; Dijkstra = A* with estimate 0

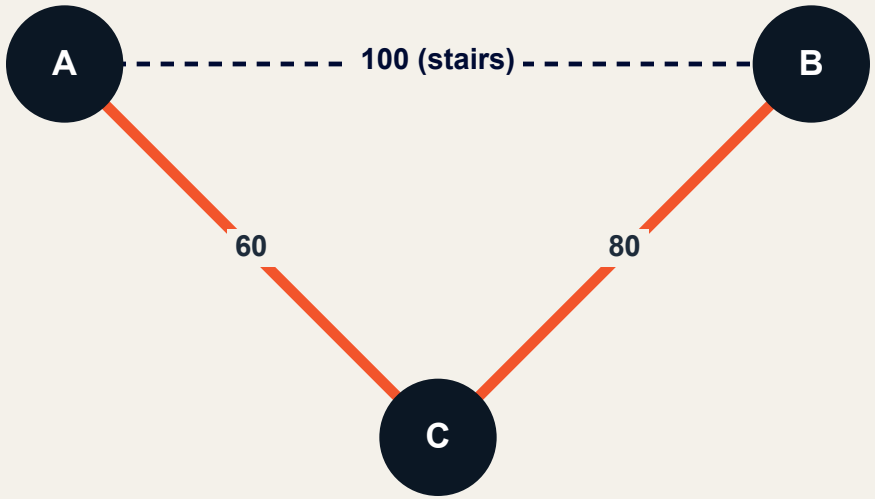
SPEAKER NOTES

This slide wins viva questions. BFS counts edges, not metres. Naive greedy commits to the locally shortest edge and can't undo it. Nuance: Dijkstra is itself a greedy algorithm in textbooks, but its greedy choice is GLOBAL (closest frontier node) and provably safe with non-negative weights. Demo: Algorithm panel → Compare.

Accessibility-Aware Routing

Accessibility changes the graph, not just the icons

- Modes: Shortest · Fastest · **Wheelchair** · Step-free · Avoid stairs.
- Before relaxing an edge, Dijkstra checks it: stairs / not step-free / not accessible → **skipped**.
- The accessible route may be **longer** — that is correct.
- Unknown accessibility is **never** shown as accessible — labelled "unverified".
- User-verified: Block A & B fronts accessible; **Block C front is not** → app says so, no fake route.



Normal: A → B = 100 m **Wheelchair: A → C → B = 140 m**

SPEAKER NOTES

Pipeline: all edges → accessibility filter → eligible edges → Dijkstra. Example from our tests: the stairs edge is removed, so the wheelchair route is 140 m instead of 100 m. Point out the Block C rule: we refuse to substitute a normal route when no verified accessible entrance exists.

Live Navigation & Simulation

Real location vs a clearly-labelled demo



- **LIVE**: shows accuracy (±m); fixes worse than ±35 m are not used; GPS jumps ignored.
- **Map matching**: snap the GPS dot to the nearest point on the route.
- **Rerouting**: > max(20 m, 1.5 × accuracy) off-route twice → re-run Dijkstra from the nearest node.
- **Arrival**: within 10 m of the destination with good accuracy — by position, not animation.
- **SIMULATION**: virtual walker on the real route; 1× = real walking time (1.4 m/s); always labelled.

SPEAKER NOTES

Honesty principle: simulation is never presented as GPS, and the raw GPS dot with its accuracy circle is shown separately from the smoothed marker. ETA = remaining distance / walking speed and is always shown as an estimate (A → Canteen ≈ 125 s ≈ 2 min).

3D Visualisation & Voice

Presentation layers on top of the algorithm

- **Globe → campus**: Earth rotates to India, camera dives, crossfades into the 3D campus.
- **3D campus**: blocks, hostels, canteen, football ground + track, courts, parking.
- **Route**: follows the real curved walkway shapes; walked part fades, next part stays bright.
- **Voice commands**: "Take me to the badminton court" → intent → Dijkstra. Ambiguous ("hostel") → asks.
- **Read Aloud**: distance, ETA, steps, accessibility warnings — generated from the route.

SUGGESTED VISUAL

Screenshot of the 3D campus with the orange route and the Navigation panel.

Second screenshot: the globe mid-flight over India.

SPEAKER NOTES

Stress separation of concerns: delete the 3D layer and Dijkstra returns the same path. Voice never computes routes — it fills in From/To/Mode. Speech recognition needs Chrome/Edge; a typed command box always works.

Technology Stack

What each tool does

Tool	Role in our project
TypeScript	JavaScript with types — catches mistakes before running
React	builds the UI from reusable components
Vite	dev server + production build
Three.js (WebGL)	draws the 3D globe and campus on the GPU
React Three Fiber + drei	3D objects as React components; orbit controls; route lines
Tailwind CSS · Framer Motion	styling · UI animations
Natural Earth (world-atlas) + d3-geo	original globe texture from public-domain country shapes
Browser APIs	Geolocation (GPS), Web Speech (voice in/out), localStorage (settings)
Vitest	71 automated tests
Our own code	**Graph, MinHeap, Dijkstra, Yen's alternatives, accessibility filter, map matching** — no graph library

SPEAKER NOTES

Key message: the frameworks are the face; the algorithmic core is written from scratch — Graph, MinHeap and Dijkstra are our own code, not a library call.

Testing & Correctness

71 automated tests · 7 test files

- Guide examples: A→B 10, A→C 3, C→B 2 □ **A → C → B = 5**.
- Unreachable, same start/end, equal-cost ties, invalid node, negative weight rejected.
- **30 random graphs**: Dijkstra checked against Floyd–Warshall (a different algorithm).
- Accessibility: wheelchair avoids stairs, lift unavailable, ramp closed, no accessible route, accessible entrance.
- Real campus: A → Canteen = 174.83 m; no direct Girls↔Boys Hostel edge; BFS/greedy never beat Dijkstra.
- Navigation: map matching, rerouting, GPS outliers; voice intents and clarification.

SPEAKER NOTES

Run `npm test` live if time allows. The Floyd–Warshall cross-check is strong evidence: two independent algorithms agree on every pair in 30 random graphs. Floyd–Warshall is also our Dynamic Programming example.

Algorithmic Paradigms — Honest Mapping

What is in the project, and what is not

Paradigm	In project?	Where
Graph	Yes — core	campus graph + Dijkstra
Greedy	Yes	Dijkstra's safe global greedy choice; naive greedy baseline (fails 173/210)
Dynamic Programming	Yes — in tests	Floyd–Warshall: d[i][j] = min(d[i][j], d[i][k] + d[k][j])
Divide & Conquer	Not used	possible: split a large campus into zones joined at gateway nodes
Backtracking	Not used	could enumerate all simple paths — exponential, unsuitable
Branch & Bound	Not used	could prune paths longer than the best found; A* is related

SPEAKER NOTES

Never claim Dijkstra is every paradigm. Being precise here impresses judges more than over-claiming.

Limitations & Future Work

What we'd do next

- Distances are **map-measured** → walk them with a measuring wheel and mark VERIFIED.
- Outdoor accessibility (kerbs, ramps, slopes) **not yet surveyed**.
- **Indoor** navigation modelled (2 west + 3 east classrooms, HOD/Staff south) but disabled until room numbers, stairs, lifts and distances are collected.
- Court/parking outlines approximate (satellite imagery predates the courts).
- Phone GPS is several metres accurate — indoor positioning would need BLE beacons / QR checkpoints.

SPEAKER NOTES

Raising limitations yourself shows understanding. Everything listed can be added by editing data files — no algorithm changes needed.

Live Demo Script

About 3 minutes

1. Search "MRTC" → **Navigate to MRTC** → globe flight → 3D campus.
2. From **Block A** → To **Canteen** → Calculate → 174.83 m via B and C.
3. **Algorithm** panel → Play (watch C relax 96.73 → 90.05) → Compare (BFS / Greedy).
4. Switch to **Wheelchair** → To **Block C** → explicit "no verified accessible entrance".
5. Voice or typed command: "take me to the hostel" → asks which one → "box cricket" → route.
6. **Simulate** at 10× with Follow → arrival detected.

SPEAKER NOTES

Rehearse this order. If the network is slow, skip the globe with Reduce motion (settings). Keep the Data panel ready in case judges ask where distances come from.

Q&A — Likely Judge Questions

Short answers to rehearse

Question	Answer in one line
Why Dijkstra?	Weighted graph, non-negative metres, need minimum total distance.
What is relaxation?	If dist[u] + w < dist[v], update dist[v] and prev[v] = u.
Why is a popped node final?	It's the smallest; any other route passes a farther node and only adds metres.
Why not BFS?	It minimises edges, not metres — A→C 96.73 m vs 90.05 m.
Where does log V come from?	Binary heap push/pop on a tree ~log V tall.
How do you get the route?	Follow prev[] from destination back to start, then reverse.
What if a ramp is closed?	Mark the edge closed; the filter rejects it; route recalculates.
Are distances accurate?	Map-measured, labelled ESTIMATED; missing ones are disabled, never guessed.

SPEAKER NOTES

Practise answering each in under 15 seconds. The full explanations with viva questions are in docs/LEARN.md.

Thank you

Questions?

github.com/vidyaasagaar/campus-navigation